Tree

沈威宇

November 13, 2026

1 Tree

I Tree

i Tree

A tree is an abstract data type of a set of nodes. A relationship between two nodes i, j is defined with

- i is called a child of j and
- j is called a parent of i; and
- a node can have at most one parent.

If not empty, a tree is recursively defined as a distinguished node r called root and zero or more nonempty trees called subtrees, where the root of each substree is a child of r.

ii Terminology

- Siblings: Nodes sharing the same parent are called siblings. Children of a node are defined with a left to right order.
- Leaf: A node with no children is called a leaf, external node, or exterior node.
- Internal node: A node that is not a leaf is called an internal node or interior node.
- Depth: The number of edges from the root to a node is called the depth of the node, while the root has depth 0.
- Height: The height of a leaf is defined as 0. The height of an internal node is defined as the maximum height of its children plus 1. The maximum depth of the nodes, i.e., the height of its root, is called the height or depth of the tree and denoted h. The height or depth of an empty tree is defined as -1. Some define heights of nodes and trees as the definition here plus 1, which is not used here.
- Ancestors: The ancestors of a node is recursively defined as itself for root and itself and the ancestors of its parent for non-root nodes.
- Descendants: The descendants of a node is recursively defined as itself for leaves and itself and the descendants of its children for internal nodes.
- Level: Nodes of the same depth are called a level. Levels are ordered by depth. The left-to-right order of nodes in a level is defined recursively to, if they are siblings, follow that of siblings; otherwise follow that of their parents.
- Subtree: A tree consisting of all descendants of a node is called a subtree. The left-to-right order of subtrees whose roots are of the same depth are defined as

that of their roots. A subtree whose root is a child of a node is called a subtree of the node or rooted at the node.

- The umber of nodes in a tree is denoted n.

iii Tree representation

A representation of tree where each pair of parent and child is connected with a edge.

iv Left child, right sibling representation

A representation of tree where each internal node and its leftmost child is connected with a edge, and each sibling that is not the leftmost one is called to the sibling at its left.

v Skewed

A tree whose every level contains only one node is called skewed.

II n-ary tree

i n-ary tree

A tree in which no node can have more than n children is called an n-ary tree. A child is indexed into one and only one integer in $[1, n]$ even if its parent has less than n children.

ii Full, proper, or plane

An n-ary tree where each internal node has exactly n children is called full, proper, or plane.

iii Filled

A level of depth $k \in \mathbb{N}$ of an n-ary tree is called filled iff each node in the level of depth $k - 1$ has exactly n children. The level of depth 0 is called filled iff the tree is nonempty.

iv Complete

An n-ary tree where every level, except possibly the last, is filled and whose nodes in the last level are as far left as possible, is called complete.

A complete n-ary tree is indexed from top to bottom and then from left to right, with the root as node 1.

v Perfect

A complete n-ary tree where all leaves are of the same depth is called perfect.

A perfect n-ary tree with height k has $\sum_{j=0}^{k} n^j$ nodes.

III Binary tree

i Binary tree

In a binary tree, the two possible children of a node are called left child and right child. A child is either left or right but not both even if it has no sibling. The left and right subtrees of a node x are denoted left(x) and right(x) respectively.

ii Complete binary tree

For a complete binary tree with n nodes:

- Parent of node i is node $\left\lfloor \frac{i}{2} \right\rfloor$ for all integers $i > 1$.
- The left child of node i is $2i + 1$ for all integers $i \leq \frac{n-1}{2}$.
- The right child of node i is $2i$ for all integers $i \leq \frac{n}{2}$.

iii Perfect binary tree

A perfect binary tree with height k has $2^{k+1} - 1$ nodes.

iv Balanced tree

A type of binary tree with height $O(\log n)$ where n is the number of nodes is called balanced.

v Balance factor (bf)

The height of the left subtree minus the height of the right subtree of a node in a binary tree is called the balance factor of the node.

vi Left-skewed and right-skewed

A binary tree whose every internal node has left child and does not have right child is called left-skewed.

A binary tree whose every internal node has right child and does not have left child is called right-skewed.

IV Tree traversal

They are defined recursively. Suppose the root is r.

i Preorder traversal

First visit r, and then visit the subtrees of r from left to right.

ii Postorder traversal

Visit the subtrees of r from left to right, and finally visit r.

iii Inorder traversal (binary tree)

First visit the left subtree of r, and then visit r, and then visit the right subtree of r.

iv Full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals

A full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a full binary tree from:
 *   P - preorder array (size num)
 *   Q - postorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *Q, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  Node *root = createNode(P[0]);
  int leftRootVal = P[1];
  int k = 0;
  while (Q[k] != leftRootVal)
```

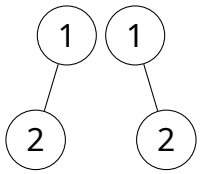
```
    k++;
  int leftSize = k + 1;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, Q, leftSize);
  root->right = construct(P + 1 + leftSize, Q + leftSize, rightSize);
  return root;
}
```

□

v Binary tree with distinct values cannot be uniquely determined by its preorder and postorder traversals

The statement that a binary tree with distinct values can be uniquely determined by its preorder and postorder traversals is false.

Proof. No. A counterexample is the following two binary trees



Both trees have preorder traversal 1, 2, and postorder traversal 2, 1, but they are different. □

vi Binary tree with distinct values can be uniquely determined by its preorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its preorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
```

```
 *   P - preorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *I, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  int rootVal = P[0];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, I, leftSize);
  root->right = construct(P + 1 + leftSize, I + leftSize + 1, rightSize);
  return root;
}
```

□

vii Binary tree with distinct values can be uniquely determined by its postorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its postorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
 *   Q - postorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *Q, int *I, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(Q[num - 1]);
```

```
  int rootVal = Q[num - 1];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
  int rightSize = num - 1 - k;
  root->left = construct(I, Q, leftSize);
  root->right = construct(I + k + 1, Q + leftSize, rightSize);
  return root;
}
```

□

V Binary search tree (BST)

i Binary search tree (BST)

A binary search tree stores keys in the nodes of a binary tree so that searching, insertion and deletion can be done efficiently. For every node x, all the keys in its left subtree are smaller than x and all keys in its right subtree are larger than x. Thus, the values are distinct.

ii Searching

Suppose we are searching for x and the root is r.

- If $x = r$, then we are done.
- If $x < r$, then we search in the left subtree.
- If $x > r$, then we search in the right subtree.

The time complexity is $O(h)$.

iii Finding minimum and maximum

To find the node containing the smallest key, start at root and go to the left child as long as there is one, and the leaf we stop at is the smallest key. Similarly for find the largest key.

The time complexity is $O(h)$.

iv Insertion

Suppose we want to insert x. First search for it, if we found it, do noting (or update something); otherwise, if we stop at y,

- if $x > y$, insert x as the right child of y if $x > y$, and
- if $x < y$, insert x as the left child of y.

The time complexity is $O(h)$.

v Deletion

Suppose we want to delete x. First search for it, if we don't found it, do noting (or update something); otherwise,

- if it is a leaf, delete it,
- if it has one child y, adjust y as a child of the parent of x of the same position as x and delete x, and
- if it has two children, choose either of the policies:
  - inorder successor: replace the value of x with the value of the minimum element y in the right subtree of x, i.e. inorder successor, i.e., the leftmost descendant of the right child of x, which has no left child but may have right child, and
    - remove node y if it has no child, and
    - replace node y with its right child if it has one;
  - inorder predecessor: replace the value of x with the value of the maximum element y in the left subtree of x, i.e. inorder predecessor, i.e., the rightmost descendant of the left child of x, which has no right child but may have left child, and
    - remove node y if it has no child, and
    - replace node y with its left child if it has one;

The time complexity is $O(h)$.

vi (Simple) rotation

A (simple) rotation on an edge between a node x and its parent p, also called rooted at or on p, with p called root and x called pivot of the rotation, is a tree operation on BST and has the following two symmetric variants:

- Right rotation: This is done when x is a left child of p. We perform the following assignments:

$$\text{left}(p) = \text{right}(x)$$

$$\text{right}(x) = p$$

- Left rotation: This is done when x is a right child of p. We perform the following assignments:

$$\text{right}(p) = \text{left}(x)$$

$$\text{left}(x) = p$$

vii Uniqueness given preorder traversal

A BST is uniquely determined by its preorder traversal.

Proof. A BST with one node is obviously unique. The first element must be root, the second element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

viii Uniqueness given postorder traversal

A BST is uniquely determined by its postorder traversal.

Proof. A BST with one node is obviously unique. The last element must be root, the first element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

ix Binary tree is BST iff inorder traversal is in ascending order

A binary tree with distinct values is a binary search tree iff its inorder traversal is sorted in ascending order.

Proof. Only if: Prove by mathematical induction. The inorder traversal of a BST with one node is sorted in ascending order. Assume the inorder traversal of any BST with fewer than n nodes of distinct values is sorted in ascending order. Consider any BST with n nodes, the inorder traversal visits the left subtree, which is smaller than root and itself sorted in ascending order since it's a BST with $n - 1$ nodes, and then root, and then the right subtree, which is larger than root and itself sorted in ascending order since it's a BST with $n - 1$ nodes. Thus, we conclude that the inorder traversal of any BST with n nodes is sorted in ascending order. By mathematical induction, the statement holds.

If: Prove by mathematical induction. A binary tree with one node is a BST. Assume that any binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order is a BST. Consider any binary tree with n nodes of distinct values and root r whose inorder traversal is sorted in ascending order. The subsequence before r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are smaller than r. The subsequence after r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are larger than r. Since the left subtree is a BST and smaller than root and the right subtree is a BST and larger than root, we conclude that any binary tree with n nodes of distinct values whose

inorder traversal is sorted in ascending order is a BST. By mathematical induction, the statement holds. □

x Number of distinct BST shapes

The number T_n of distinct shapes of BSTs with n nodes is:

$$T_0 = 1$$

$$T_n = \sum_{i=0}^{n-1} T_i T_{n-1-i}, \quad n \in \mathbb{N}$$

$$T_n = \frac{1}{(n+1)} \binom{2n}{n}, \quad n \in \mathbb{N}_0$$

Proof. Obviously

$$T_0 = T_1 = 1$$

Let the values be

$$k_1 < k_2 < \ldots < k_n$$

Suppose we choose k_i as root, where $1 \leq i \leq n$. The left subtree contains $k_1, \ldots, k_{i-1}$, so it has $i-1$ nodes and T_{i-1} possible combinations. The right subtree contains $k_{i+1}, \ldots, k_n$, so it has $n-1$ nodes and T_{n-i} possible combinations.

Summing over all choices gives

$$T_n = \sum_{i=1}^{n} T_{i-1} T_{n-i} = \sum_{i=0}^{n-1} T_i T_{n-1-i}$$

Using generating function:

$$C(x) = \sum_{n=0}^{\infty} T_n x^n$$

$$C(x) = 1 + xC(x)^2$$

$$C(x) = \frac{1 \pm \sqrt{1-4x}}{2x}$$

$$C(0) = \mp \frac{1}{\sqrt{1-4x}} = T(0) = 1$$

$$C(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

$$
\begin{aligned}
\sqrt{1-4x} &= \sum_{k=0}^{\infty} \binom{0.5}{k} (-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{\prod_{i=0}^{k-1}(0.5-i)}{k!}(-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{2^k \prod_{i=0}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{2k-2} i}{k! \prod_{i=1}^{k-1} 2i}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k (2k-2)!}{k! 2^{k-1}(k-1)!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2(2k-2)!}{k!(k-1)!}x^k
\end{aligned}
$$

$$
\begin{aligned}
C(x) &= \frac{1-\sqrt{1-4x}}{2x} \\
&= \sum_{k=1}^{\infty} \frac{(2k-2)!}{k!(k-1)!}x^{k-1} \\
&= \sum_{n=0}^{\infty} \frac{(2n)!}{(n+1)!n!}x^n \\
&= \sum_{n=0}^{\infty} \frac{1}{(n+1)}\binom{2n}{n}x^n
\end{aligned}
$$

$$
T_n = \frac{1}{(n+1)}\binom{2n}{n}, \quad n \in \mathbb{N}_0
$$

□

VI AVL tree (Adelson-Velsky and Landis tree)

i AVL tree (Adelson-Velsky and Landis tree)

An AVL tree is a self-balancing binary search tree (BST) that keeps info of height, invented by Georgy Adelson-Velsky and Evgenii Landis. An AVL tree needs to comply with the property that for every node with two children in the tree, the height of its left and right subtrees differ by at most 1.

Some define heights of nodes and AVL trees as the definition used here plus 1, which is not used here.

ii Relation of height and numbers of nodes

Let the minimum number of nodes in an AVL tree of height h be N_h. Since the minimum numbers of subtrees height of the root of an AVL tree of height h are $h-1$ and $h-2$, we have:

$$N_{-1} = 0$$

$$N_0 = 1$$

$$N_h = N_{h-1} + N_{h-2} + 1, \quad h \in \mathbb{N}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1, \quad h+1 \in \mathbb{N}_0$$

Proof. Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2 + \sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{7+3\sqrt{5}}{2} = \varphi^4 = 3\varphi + 2$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$\frac{3-\sqrt{5}}{2} = \frac{1}{\varphi^2} = 1 - \frac{1}{\varphi} = 2 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$N_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$N_0 = a + b + c = 1$$

$$N_1 = a\varphi - \frac{b}{\varphi} + c = 0 + 1 + 1 = 2$$

$$N_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 1 + 2 + 1 = 4$$

$$N_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 4$$

$$N_2 - N_1 = a + b = 4 - 2 = 2$$

$$c = N_0 - (a + b) = 1 - 2 = -1$$

$$N_1 - c = a\varphi - \frac{2-a}{\varphi} = 2 + 1 = 3$$

$$a\left(\varphi^2 + 1\right) = 3\varphi + 2$$

$$a = \frac{3\varphi + 2}{\varphi + 2} = \frac{\varphi^4}{\varphi + 2}$$

$$b = 2 - a = \frac{2-\varphi}{\varphi+2} = \frac{1}{\varphi^2(\varphi+2)}$$

$$N_h = \frac{1}{\varphi+2}\left(\varphi^{h+4} + \left(-\frac{1}{\varphi}\right)^{h+2}\right) - 1$$

$$\frac{1}{\varphi+2} = \frac{2}{5+\sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1$$

□

So

$$N_h = \Theta(\left(\frac{1+\sqrt{5}}{2}\right)^h)$$

$$h = \log_{\frac{1+\sqrt{5}}{2}} N_h + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Let the maximum number of nodes in an AVL tree of height h be M_h. Since the maximum numbers of subtrees height of the root of an AVL tree of height h are $h-1$, we have:

$$M_{-1} = 0$$

$$M_h = 2M_{h-1} + 1, \quad h \in \mathbb{N}_0$$

$$M_h = 2^{h+1} - 1, \quad h + 1 \in \mathbb{N}_0$$

So

$$M_h \in \Theta(\log_2 M_h)$$

$$h = \log_2 M_h + O(1)$$

Let the number of nodes be n.

$$N_h \le n \le M_h$$

$$\log_2 n + O(1) \le h \le \log_{\frac{1+\sqrt{5}}{2}} n + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Thus

$$h \in \Theta(\log n)$$

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in AVL tree is the same as in BST.

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

iv Rotation

When insertion/deletion, the AVL property may be violated, called unbalanced. Since an insertion/deletion involves adding/deleting a single node, this can only increase/decrease the height of some subtree by 1. Thus, if the AVL property is violated at a node x, it means that the heights of left and right subtrees of it differ by exactly 2. We need to transform the tree to restore the AVL property, called rebalancing, using a single/simple rotation or a double rotation, each has two symmetric variants:

- LL case / right single rotation: Let the left child of x be y. The LL case is where the left subtree of y has height $h+1$, while the right subtree of y and the right subtree of x have height h. We perform a right rotation on the edge between y and x, i.e.,

$$\text{left}(x) = \text{right}(y)$$
$$\text{right}(y) = x$$

- RR case / left single rotation: Let the right child of x be y. The RR case is where the right tree of y has height $h+1$, while the left subtree of y and the left subtree of x have height h. We perform a left rotation on the edge between y and x, i.e.,

$$\text{right}(x) = \text{left}(y)$$
$$\text{left}(y) = x$$

- LR case / left-then-right double rotation: Let the left child of x be y and the right child of y be z. The LR case is where the right subtree of y has height $h+1$, while the left subtree of y and the right subtree of x have height h. We perform a left rotation on the edge between z and y and then a right rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{right}(y) = \text{left}(z)$$
$$\text{left}(z) = y$$
$$\text{left}(x) = \text{right}(z)$$
$$\text{right}(z) = x$$

- RL case / right-then-left double rotation: Let the right child of x be y and the left child of y be z. The RL case is where the left subtree of y has height $h+1$, while the right subtree of y and the left subtree of x have height h. We perform a right rotation on the edge between z and y and then a left rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{left}(y) = \text{right}(z)$$
$$\text{right}(z) = y$$
$$\text{right}(x) = \text{left}(z)$$
$$\text{left}(z) = x$$

v Insertion

Insertion in AVL tree involves first doing insertion as with a normal BST, and then tracing the path from the new leaf to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then break the loop since no second violation may exist.

Proof. Let x be the first node encountered when traversing upward from the newly inserted node toward the root that violates the AVL property. Since an insertion involves adding a single node, this can only increase the height of some subtree by 1. Thus, the heights of the left and right subtrees differ by exactly 2. For any of the four cases (LL, RR, LR, RL), since the inserted node must be in the subtree of height $h+1$ of y, the height of the subtree whose root is x has height $h+2$ both before insertion and after rotation. Thus, the fact that the AVL property is satisfied for all ancestors of x before insertion implies that the AVL property is satisfied for all ancestors of x after rotation. □

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

vi Deletion

Deletion in AVL tree involves first doing deletion the same as in BST, and then tracing the path from

- the original parent of the deleted node if the deleted node had zero or one child, and
- the original parent of the inorder successor or inorder predecessor that was moved if the deleted node had two children,

to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then proceed to the parent of x since violation may propagate upwards.

The worst-case and amortized time complexities are $O(h) = O(\log n)$.

vii Number of distinct AVL tree shapes

The number $A_{h,n}$ of distinct shapes of AVL trees with n nodes and height h is:

$$A_{-1,0} = 1$$

$$A_{-1,n} = 0, \quad n \neq 0$$

$$A_{0,1} = 1$$

$$A_{0,n} = 0, \quad n \neq 1$$

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}, \quad h \in \mathbb{N}, n \in \mathbb{N}_0$$

Proof. The root uses one node, leaving $n-1$ nodes for the two subtrees. Since the whole tree has height h, the two trees must either have heights $h-1, h-2$ or both have height $h-1$. Thus,

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}$$

□

viii Number of distinct AVL tree shapes with minimum number of nodes given height

The number B_h of distinct shapes of AVL trees with height h and minimum number of nodes is:

$$B_{-1} = 1$$

$$B_0 = 1$$

$$B_h = 2B_{h-1}B_{h-2}, \quad h \in \mathbb{N}$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right)-1}, \quad h+1 \in \mathbb{N}_0$$

Proof. To minimize the number of nodes, the two subtrees of root must be the AVL tree with height $h-1, h-2$ and minimum number of nodes, which have B_{h-1}, B_{h-2} distinct shapes respectively. There are two possible arrangements:

- Left subtree has height $h-1$, and right subtree has height $h-2$.
- Left subtree has height $h-2$, and right subtree has height $h-1$.

Thus,

$$B_h = 2B_{h-1}B_{h-2}$$

Let

$$C_h = \log_2 B_h$$

$$C_h = C_{h-1} + C_{h-2} + 1$$

Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2+\sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$C_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$C_0 = a + b + c = \log_2 1 = 0$$

$$C_1 = a\varphi - \frac{b}{\varphi} + c = \log_2(2 * 1 * 1) = 1$$

$$C_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 0 + 1 + 1 = 2$$

$$C_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 2$$

$$C_2 - C_1 = a + b = 2 - 1 = 1$$

$$c = C_0 - (a + b) = 0 - 1 = -1$$

$$C_1 - c = a\varphi - \frac{1-a}{\varphi} = 1 + 1 = 2$$

$$a\left(\varphi^2 + 1\right) = 2\varphi + 1$$

$$a = \frac{2\varphi + 1}{\varphi + 2} = \frac{\varphi^3}{\varphi + 2}$$

$$b = 1 - a = \frac{1-\varphi}{\varphi+2} = -\frac{1}{\varphi(\varphi+2)}$$

$$C_h = \frac{1}{\varphi+2}\left(\varphi^{h+3} + \left(-\frac{1}{\varphi}\right)^{h+1}\right) - 1$$

$$\frac{1}{\varphi+2} = \frac{2}{5+\sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$C_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right) - 1$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right)-1}$$

□

VII Splay tree

i Splay tree

A splay tree is a self-adjusting binary search tree (BST) that keeps no info of height and in which every access operation moves the accessed node to the root through an operation called splaying, invented by Daniel Dominic Sleator and Robert Endre Tarjan. The idea is locality, that is, if an item is accessed, it is likely to be accessed again soon. The worst-case searching, insertion, and deletion time complexities are as with a normal BST, while the amortized time complexities are $O(\log n)$.

ii (Bottom-up) splay

When a node x is accessed, a (bottom-up) splay operation consisting of a sequence of splay steps is performed on x to move it to the root, called splaying x. Let the parent of x be p and the parent of p be g. There are three types of splay steps, each of which has two symmetric variants:

- Zig step / simple rotation: This is done if x is a child of the root. We perform a rotation on the edge between x and root. This only happens as the last step and only happens when x originally has odd depth.
- Zig-zig step: This is done either when x is a left child of p and p is a left child of g, or when x is a right child of p and p is a right child of g. We perform a rotation on the edge between p and g and then a rotation on the edge between x and p.
- Zig-zag step: This is done either when x is a left child of p and p is a right child of g, or when x is a right child of p and p is a left child of g. We perform a rotation on the edge between x and p and then a rotation on the edge between x, now at the original position of p, and g.

A zig step is a 1-level splay step while a zig-zig or zig-zag step is a 2-level splay step.

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in splay tree involves first doing the operation as with a normal BST and then splay the found node if found.

iv Join

Given two BSTs S and T such that all elements of S are smaller than all elements of T, the following steps can be used to join them to a single BST: either

1. Splay the largest item in S. Now this item is in the root r of S and has no right child.
2. Set the right child of the new root r of S to the root of T.

or

1. Splay the smallest item in T. Now this item is in the root r of T and has no left child.
2. Set the left child of the new root r of T to the root of S.

v Split

Given a BST and a value x, splitting the BST at x returns two new BSTs, one containing all elements less than or equal to x, L, and the other containing all elements greater than x, R. This can be done with:

1. Search for x in the tree. If we found it, let y be that node; otherwise let y be the node we stop at.
2. Splay y.
3. Split its right subtree, which is R, from the rest of the tree, which is L.

vi Insertion

To insert a value x into a splay tree, there are two ways: either

1. Insert x as with a normal BST.
2. Splay x.

or

1. Split at x. However, if x is found in the tree, stop after splaying x.
2. Create a new tree with x as root and the two split trees as its subtrees.

vii Deletion

To delete a node x from a splay tree: either

1. Search for x.
2. Splay x.
3. Delete x and join the two subtrees of x.

or

1. Delete x as with a normal BST.
2. Splay the original parent of the deleted x.

viii Amortized analysis

We define size of a node $s(x)$ as the number of nodes in the subtree whose root is node x, rank of a node as

$$r(x) = \log_2 s(x)$$

and potential function of a tree $P(T)$ is

$$P(T) = \sum_{x \in T} r(x)$$

The actual cost C_i of an operation i is defined as the number of rotations in the operation. Let the state before operation i be denoted by the original functions and the state after operation i be denoted with a prime symbol. The amortized cost A_i of an operation i with actual cost C_i that transforms the tree T_{i-1} to T_i is defined as

$$A_i = C_i + \Delta P = C_i + P(T_i) - P(T_{i-1})$$

- Zig step:

$$C_i = 1$$

$$r'(p) \leq r(p)$$

$$\Delta P = r'(x) + r'(p) - r(x) - r(p) \leq r'(x) - r(x)$$

$$A_i \leq 1 + r'(x) - r(x)$$

- Zig-zig step:

$$C_i = 2$$

$$r'(p) \leq r(g)$$

$$r(x) \leq r(p)$$

$$s(x) + s'(g) = s'(x) - 2 \leq s'(x)$$

The log function is concave, i.e.,

$$\frac{\log_2 a + \log_2 b}{2} \leq \log_2\left(\frac{a+b}{2}\right)$$

i.e.,

$$log_2 a + \log_2 b \leq 2\log_2(a+b) - 2$$

Hence

$$r(x) + r'(g) = 2r'(x) - 2$$

$$r'(g) \leq 2r'(x) - r(x) - 2$$

$$\Delta P = r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \leq r'(x) + r'(g) - 2r(x) \leq 3(r'(x) - r(x)) - 2$$

$$A_i \leq 3(r'(x) - r(x))$$

- Zig-zag step:

$$C_i = 2$$

$$s'(p) + s'(g) = s'(x) - 1 \leq s'(x)$$

The log function is concave, i.e.,

$$\frac{\log_2 a + \log_2 b}{2} \leq \log_2\left(\frac{a+b}{2}\right)$$

i.e.,

$$log_2 a + \log_2 b \leq 2\log_2(a+b) - 2$$

Hence

$$r'(p) + r'(g) \leq 2r'(x) - 2$$

$$r(x) \leq r(p) \leq r(g)$$

$$\Delta P = r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \leq 3(r'(x) - r(x)) - 2$$

$$A_i \leq 3(r'(x) - r(x))$$

When splaying a node x, zero or more 2-level splay steps followed by zero or one 1-level splay step is conducted. Let $r_2(x)$ be the rank of x just after the last 2-level splay step and $r_f(x)$ be the final rank of x. The amortized cost A of splaying x is

$$\begin{aligned} A &\leq 1 + r_f(x) - r_2(x) + 3(r_2(x) - r(x)) \\ &\leq 1 + 3(r_f(x) - r_2(x)) + 3(r_2(x) - r(x)) \\ &= 1 + 3(r_f(x) - r(x)) \\ &\leq 1 + 3(\log_2 n - r(x)) \\ &= O(\log n) \end{aligned}$$

For a sequence of m accesses in a splay tree T, we can bound the total actual cost with

$$\sum_{i=1}^{m} C_i = P(T_0) - P(T_m) + \sum_{i=1}^{m} A_i$$

For every node x,

$$1 \leq s(x) \leq n$$

$$0 \leq r(x) \leq \log_2 n$$

Thus the change of rank $\Delta r(x)$ of a node x is

$$\Delta r(x) \leq \log_2 n$$

Thus the change of potential function is

$$P(T_0) - P(T_m) \leq \sum_{x \in T} \log_2 n = O\left(n \log n\right)$$

ix Weighted analysis

The above analysis can be generalized by assigning each node x a weight $w(x) \geq 1$ and defining $s(x)$ as the sum of weights of nodes in the subtree whose root is node x, called weighted analysis. Note that the previous analysis corresponds to the special case $w(x) = 1$. The sum of all weights is denoted W. The same analysis gives:

$$A \leq 1 + 3(\log_2 n - r(x)) = O(\log\left(\frac{W}{w(x)}\right) + 1)$$

For a sequence of m accesses in a splay tree T, we can bound the total actual cost with

$$\sum_{i=1}^{m} C_i = P(T_0) - P(T_m) + \sum_{i=1}^{m} A_i$$

For every node x,

$$w(x) \leq s(x) \leq W$$

$$\log_2(w(x)) \leq r(x) \leq \log_2 W$$

Thus the change of rank $\Delta r(x)$ of a node x is

$$\Delta r(x) \leq \log_2\left(\frac{W}{w(x)}\right)$$

Thus the change of potential function is

$$P(T_0) - P(T_m) \leq \sum_{x \in T} \log_2\left(\frac{W}{w(x)}\right) = O\left(\sum_{x \in T} \log\left(\frac{W}{w(x)}\right)\right)$$

x Balance theorem

The cost of performing a sequence of m accesses in a splay tree containing n nodes is

$$O(m \log n + n \log n)$$

Proof. Take a constant weight, e.g.,

$$w(x) = 1$$

Then

$$W = n$$

$$A_i = O(\log n)$$

$$P(T_0) - P(T_m) = O(n \log n)$$

□

This theorem implies that splay trees perform at least as well as AVL trees on sequences of at least n accesses.

xi Static optimality theorem

The cost of performing a sequence of m accesses in a splay tree T containing n nodes, with each node accessed at least once and q_x being the number of times node x is accessed, is

$$O\left(m + \sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right)\right)$$

Proof. Let

$$w(x) = q_x$$

Then

$$W = m$$

$$A_i = O\left(\log\left(\frac{m}{q_x}\right) + 1\right)$$

$$\sum_{i=1}^{m} A_i = O\left(\sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right) + m\right)$$

$$q_x \geq 1$$

$$P(T_0) - P(T_m) = O\left(\sum_{x \in T} \log\left(\frac{m}{q_x}\right)\right) = O\left(\sum_{x \in T} q_x \log\left(\frac{m}{q_x}\right)\right)$$

□

xii Working set theorem

The cost of performing a sequence S of m accesses in a splay tree T containing n nodes, with $t(x)$ being the number of distinct nodes accessed before the previous time node x was accessed for each $x \in S$, is

$$O\left(m + n \log n + \sum_{x \in S} \log(t(x) + 1)\right)$$

Proof. Let

$$w(x) = (t(x) + 1)^{-2}$$

Then

$$W = O(1)$$

$$A_i = O\left(\log(t(x) + 1) + 1\right)$$

$$\sum_{i=1}^{m} A_i = O\left(\sum_{x \in S} \log(t(x) + 1) + m\right)$$

$$P(T_0) - P(T_m) = O\left(n \log n\right)$$

□

xiii Locality property or strong balance theorem

The cost of performing a sequence of m accesses in a splay tree containing n nodes, with k distinct nodes accessed, is

$$O(m + m \log k + n \log n)$$

Proof. Use working set theorem.

$$t(x) = O(k)$$

$$O\left(m + n \log n + \sum_{x \in S} \log(t(x) + 1)\right) = O(m + n \log n + m \log k)$$

□

xiv Top-down splay

An alternative implementation of splay that is done together with searching. Suppose we want to splay x. We start at the root and maintain two trees which are initially empty, left tree L of nodes smaller than x with current node r and right tree R of nodes larger than x with current node l, during the traversal with current node m, which is initially the root of the tree, where the subtree whose root is m is called middle tree M. In each step, there are three cases, each with two symmetric variants:

- $x <=$ left(m): We perform a rotation on the edge between m and left(m), and assign the latter, node the parent of m, to m, and then remove the parent relation of m.
- $x >=$ right(m): We perform a rotation on the edge between m and right(m), and assign the latter, node the parent of m, to m.
- left(m) $< x < m$: We connect m as left(r) (or, if R is empty, the root of R and remove the left child relation of R at the end of this step), assign m to r, and assign left(m) to m, and then remove the parent relation of m.
- right(m) $> x > m$: We connect m as right(l) (or, if L is empty, the root of L and remove the right child relation of L at the end of this step), assign m to l, and assign right(m) to m, and then remove the parent relation of m.
- $x = m$, $x < m$ and m has no left child, or $x > m$ and m has no right child: The traversal stops at m. We take m as the new root, and then
  - if L is nonempty, connect left(m) as right(l), and connect the root of L as left(m);
  - if R is nonempty, connect right(m) as left(r), and connect the root of R as right(m).

Top-down splay has the same worst-case and amortized searching, insertion, and deletion time complexities as bottom-up splay and $O(1)$ extra space.

VIII An implementation of binary tree, BST, ASL tree, and splay tree in C++

```cpp
#pragma once
#include <algorithm>
#include <cctype>
#include <concepts>
#include <cstddef>
#include <functional>
#include <istream>
#include <ostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

template <class T, class Node> class BaseBinaryTreeNode {
public:
  using value_type = T;
  using node_type = Node;

  T value;
  Node *parent;
  Node *left = nullptr;
  Node *right = nullptr;

  BaseBinaryTreeNode(const T &_value = T(), Node *_parent = nullptr)
      : value(_value), parent(_parent) {}

  virtual ~BaseBinaryTreeNode() = default;
};

template <class T>
class BinaryTreeNode : public BaseBinaryTreeNode<T, BinaryTreeNode<T>> {
public:
  using Base = BaseBinaryTreeNode<T, BinaryTreeNode<T>>;
  using Base::Base;
  using value_type = T;
  using node_type = Base::node_type;
};

template <class T, class Node = BinaryTreeNode<T>> class BinaryTreeStatic {
public:
  using node_type = Node;

  static std::vector<T> preorder(Node *node) {
    if (node == nullptr)
      return std::vector<T>();
    std::vector<T> result;
    std::vector<Node *> stack;
    stack.push_back(node);
    while (!stack.empty()) {
      Node *cur = stack.back();
      stack.pop_back();
      result.push_back(cur->value);
      if (cur->right != nullptr)
        stack.push_back(cur->right);
      if (cur->left != nullptr)
        stack.push_back(cur->left);
```

```
    }
    return result;
  }

  static std::vector<Node *> preorder_node(Node *node) {
    if (node == nullptr)
      return std::vector<Node *>();
    std::vector<Node *> result, stack;
    stack.push_back(node);
    while (!stack.empty()) {
      Node *cur = stack.back();
      stack.pop_back();
      result.push_back(cur);
      if (cur->right != nullptr)
        stack.push_back(cur->right);
      if (cur->left != nullptr)
        stack.push_back(cur->left);
    }
    return result;
  }

  static std::vector<T> inorder(Node *node) {
    if (node == nullptr)
      return std::vector<T>();
    std::vector<T> result;
    std::vector<Node *> stack;
    while (!stack.empty() || node != nullptr) {
      while (node != nullptr) {
        stack.push_back(node);
        node = node->left;
      }
      node = stack.back();
      stack.pop_back();
      result.push_back(node->value);
      node = node->right;
    }
    return result;
  }

  static std::vector<Node *> inorder_node(Node *node) {
    if (node == nullptr)
      return std::vector<Node *>();
    std::vector<Node *> result, stack;
    while (!stack.empty() || node != nullptr) {
      while (node != nullptr) {
        stack.push_back(node);
        node = node->left;
      }
      node = stack.back();
      stack.pop_back();
      result.push_back(node);
      node = node->right;
    }
    return result;
  }

  static std::vector<T> postorder(Node *node) {
    if (node == nullptr)
      return std::vector<T>();
    std::vector<T> result;
    std::vector<Node *> stack;
    Node *last_visited = nullptr;
```

```
121     while (!stack.empty() || node != nullptr) {
122       if (node != nullptr) {
123         stack.push_back(node);
124         node = node->left;
125       } else {
126         Node *right = stack.back()->right;
127         if (right != nullptr && last_visited != right)
128           node = right;
129         else {
130           result.push_back(stack.back()->value);
131           last_visited = stack.back();
132           stack.pop_back();
133         }
134       }
135     }
136     return result;
137   }
138
139   static std::vector<Node *> postorder_node(Node *node) {
140     if (node == nullptr)
141       return std::vector<Node *>();
142     std::vector<Node *> result, stack;
143     Node *last_visited = nullptr;
144     while (!stack.empty() || node != nullptr) {
145       if (node != nullptr) {
146         stack.push_back(node);
147         node = node->left;
148       } else {
149         Node *right = stack.back()->right;
150         if (right != nullptr && last_visited != right)
151           node = right;
152         else {
153           result.push_back(stack.back());
154           last_visited = stack.back();
155           stack.pop_back();
156         }
157       }
158     }
159     return result;
160   }
161
162   // 1: left child
163   // 0: right child
164   // -1: node's parent is nullptr
165   // -2: node is nullptr
166   static signed char is_left_child(Node *node) noexcept {
167     if (node == nullptr)
168       return -2;
169     if (node->parent == nullptr)
170       return -1;
171     return node->parent->left == node;
172   }
173
174   static Node *leftmost_child(Node *node) {
175     if (node == nullptr)
176       return nullptr;
177     while (node->left != nullptr)
178       node = node->left;
179     return node;
180   }
181
182   static Node *rightmost_child(Node *node) {
```

```
    if (node == nullptr)
      return nullptr;
    while (node->right != nullptr)
      node = node->right;
    return node;
  }

  // LaTeX forest representation: [value[left][right]]
  // A missing child whose sibling exists is emitted as [,phantom]
  static void latex_forest(Node *node, std::ostream *oss, bool outer = true) {
    if (node == nullptr || oss == nullptr)
      return;
    if (outer)
      *oss << "\\begin{center}\n\\begin{forest}\n";
    *oss << "[" << node->value;
    bool has_left = (node->left != nullptr);
    bool has_right = (node->right != nullptr);
    if (has_left)
      latex_forest(node->left, oss, false);
    else if (has_right)
      *oss << "[,phantom]";
    if (has_right)
      latex_forest(node->right, oss, false);
    else if (has_left)
      *oss << "[,phantom]";
    *oss << "]";
    if (outer)
      *oss << "\n\\end{forest}\n\\end{center}\n";
  }
};

// Take LaTeX forest representation: [value[left][right]]
// Output root node
template <class T, class Node = BinaryTreeNode<T>>
  requires std::default_initializable<T> &&
           requires(std::istream &is, T &value) {
             { is >> value } -> std::same_as<std::istream &>;
           }
Node *from_latex_forest(std::string s) {
  std::erase_if(s, [](unsigned char c) { return std::isspace(c); });
  if (s.empty())
    return nullptr;
  std::size_t pos = 0;
  std::vector<Node *> owned;
  auto consume = [&](char expected) {
    if (pos >= s.size() || s[pos] != expected)
      throw std::invalid_argument(std::string("from_latex_forest: expected '") +
                                  expected + "'");
    ++pos;
  };
  auto parse_node = [&](auto &&self) -> Node * {
    consume('[');
    if (pos + 9 <= s.size() && s[pos] == ',' &&
        s.compare(pos + 1, 7, "phantom") == 0 && s[pos + 8] == ']') {
      pos += 9;
      return nullptr;
    }
    std::size_t start = pos;
    while (pos < s.size() && s[pos] != '[' && s[pos] != ']')
      ++pos;
    std::string value_str = s.substr(start, pos - start);
    std::istringstream vs(value_str);
```

```
    T value{};
    if (!(vs >> value))
      throw std::invalid_argument("from_latex_forest: bad value '" + value_str +
                                  "'");
    Node *node = new Node(value);
    owned.push_back(node);
    if (pos < s.size() && s[pos] == '[') {
      node->left = self(self);
      if (node->left)
        node->left->parent = node;
      node->right = self(self);
      if (node->right)
        node->right->parent = node;
    }
    consume(']');
    return node;
  };
  Node *root = nullptr;
  try {
    root = parse_node(parse_node);
    if (pos != s.size())
      throw std::invalid_argument("from_latex_forest: trailing characters");
  } catch (...) {
    for (Node *n : owned)
      delete n;
    throw;
  }
  return root;
}

template <class T, class Node = BinaryTreeNode<T>>
class BinaryTree : public BinaryTreeStatic<T, Node> {
public:
  using node_type = Node;
  using Base = BinaryTreeStatic<T, Node>;
  using Base::inorder;
  using Base::inorder_node;
  using Base::is_left_child;
  using Base::latex_forest;
  using Base::leftmost_child;
  using Base::postorder;
  using Base::postorder_node;
  using Base::preorder;
  using Base::preorder_node;
  using Base::rightmost_child;

  mutable Node *root; // Allow root->parent != nullptr
  std::ostream *oss;  // latex_forest logger

  BinaryTree(Node *_root = nullptr, std::ostream *_oss = nullptr)
      : root(_root), oss(_oss) {}

  BinaryTree(const BinaryTree &) = delete;

  BinaryTree(BinaryTree &&other) noexcept
      : root(std::exchange(other.root, nullptr)) {}

  void set_logger(std::ostream *_oss) { oss = _oss; }

  // O(n)
  std::vector<T> preorder() const { return preorder(root); }
  std::vector<Node *> preorder_node() const { return preorder_node(root); }
```

```
307   std::vector<T> inorder() const { return inorder(root); }
308   std::vector<Node *> inorder_node() const { return inorder_node(root); }
309   std::vector<T> postorder() const { return postorder(root); }
310   std::vector<Node *> postorder_node() const { return postorder_node(root); }
311
312   // O(h)
313   Node *leftmost_child() const { return leftmost_child(root); }
314   Node *rightmost_child() const { return rightmost_child(root); }
315
316   virtual void log(std::string_view msg) const {
317     if (oss != nullptr)
318       *oss << msg << '\n';
319   }
320
321   virtual void log_forest(Node *node) const {
322     if (oss != nullptr)
323       latex_forest(node, oss);
324   }
325
326   // O(n)
327   virtual void log_forest() const { return log_forest(root); }
328
329 protected:
330   // O(1)
331   virtual Node *replace(Node *node, Node *replacement,
332                         bool deleting = false) const {
333     switch (is_left_child(node)) {
334     case -2:
335       throw std::invalid_argument("Node is nullptr");
336       break;
337     case 1:
338       node->parent->left = replacement;
339       break;
340     case 0:
341       node->parent->right = replacement;
342       break;
343     }
344     if (root == node)
345       root = replacement;
346     if (replacement != nullptr)
347       replacement->parent = node->parent;
348     Node *parent = node->parent;
349     if (deleting)
350       delete node;
351     return parent;
352   }
353
354   void recursive_clear(Node *node) noexcept {
355     if (node == nullptr)
356       return;
357     recursive_clear(node->left);
358     recursive_clear(node->right);
359     delete node;
360   }
361
362 public:
363   // recursive:
364   // >0: stack
365   // 0: try stack catch recursive
366   // <0: recursive
367   void clear(Node *node, signed char recursive = 0) noexcept {
368     if (node == nullptr)
```

```
      return;
    if (root == node)
      root = nullptr;
    switch (is_left_child(node)) {
    case 1:
      node->parent->left = nullptr;
      break;
    case 0:
      node->parent->right = nullptr;
      break;
    }
    auto stack_clear = [](Node *node) {
      std::vector<Node *> stack = preorder_node(node);
      for (auto it = stack.rbegin(); it != stack.rend(); ++it)
        delete *it;
    };
    if (recursive > 0)
      stack_clear(node);
    else if (recursive == 0) {
      try {
        stack_clear(node);
      } catch (...) {
        recursive_clear(node);
      }
    } else
      recursive_clear(node);
  }

  void clear() noexcept { clear(root); }

  virtual ~BinaryTree() { clear(); }

  BinaryTree &operator=(const BinaryTree &) = delete;

  BinaryTree &operator=(BinaryTree &&other) noexcept {
    if (this != &other) {
      clear();
      root = std::exchange(other.root, nullptr);
    }
    return *this;
  }

  // O(1)
  virtual Node *rotate_right(Node *node) const {
    if (node == nullptr || node->left == nullptr)
      throw std::invalid_argument("Node is nullptr");
    Node *child = node->left;
    replace(node, child);
    node->left = child->right;
    if (child->right != nullptr)
      child->right->parent = node;
    child->right = node;
    node->parent = child;
    log("Rotate right:\n");
    log_forest();
    return child;
  }

  // O(1)
  virtual Node *rotate_left(Node *node) const {
    if (node == nullptr || node->right == nullptr)
      throw std::invalid_argument("Node is nullptr");
```

```
    Node *child = node->right;
    replace(node, child);
    node->right = child->left;
    if (child->left != nullptr)
      child->left->parent = node;
    child->left = node;
    node->parent = child;
    log("Rotate left:\n");
    log_forest();
    return child;
  }

  // O(1)
  virtual Node *rotate_parent(Node *child) const {
    if (child == nullptr || child->parent == nullptr)
      throw std::invalid_argument("Node is nullptr");
    if (is_left_child(child) == 1)
      return rotate_right(child->parent);
    return rotate_left(child->parent);
  }
};

template <class T, class Node = BinaryTreeNode<T>, class Compare = std::less<T>>
  requires std::strict_weak_order<const Compare &, const T &, const T &>
class BST : public BinaryTree<T, Node> {
public:
  using node_type = Node;
  using Base = BinaryTree<T, Node>;
  using Base::clear;
  using Base::inorder;
  using Base::inorder_node;
  using Base::leftmost_child;
  using Base::log;
  using Base::log_forest;
  using Base::postorder;
  using Base::postorder_node;
  using Base::preorder;
  using Base::preorder_node;
  using Base::rightmost_child;
  using Base::root;
  using Base::rotate_left;
  using Base::rotate_right;

  [[no_unique_address]] Compare comp;

protected:
  using Base::replace;

public:
  BST() : BST(nullptr, Compare(), nullptr) {}
  explicit BST(const Compare &_comp, std::ostream *_oss = nullptr)
      : BST(nullptr, _comp, _oss) {}
  BST(Node *_root, const Compare &_comp, std::ostream *_oss = nullptr)
      : Base(_root, _oss), comp(_comp) {}

  BST(const BST &) = delete;
  BST &operator=(const BST &) = delete;

  BST(BST &&) = default;
  BST &operator=(BST &&) = default;

  // O(h)
```

```
493   virtual Node *find_min(Node *node) const { return leftmost_child(node); }
494   virtual Node *find_max(Node *node) const { return rightmost_child(node); }
495   virtual Node *find_min() const { return find_min(root); }
496   virtual Node *find_max() const { return find_max(root); }
497
498   // Return node, found
499   // O(h)
500   virtual std::pair<Node *, bool> search(const T &x) const {
501     if (root == nullptr)
502       return {nullptr, false};
503     Node *node = root;
504     while (true) {
505       if (comp(x, node->value)) {
506         if (node->left == nullptr)
507           return {node, false};
508         node = node->left;
509       } else if (comp(node->value, x)) {
510         if (node->right == nullptr)
511           return {node, false};
512         node = node->right;
513       } else
514         return {node, true};
515     }
516   }
517
518   // O(h)
519   Node *inorder_successor(Node *node) const {
520     if (node == nullptr)
521       throw std::invalid_argument("Node is nullptr");
522     if (node->right != nullptr)
523       return leftmost_child(node->right);
524     if (node == root)
525       return nullptr;
526     Node *p = node->parent;
527     while (p != nullptr && node == p->right) {
528       if (p == root)
529         return nullptr;
530       node = p;
531       p = p->parent;
532     }
533     return p;
534   }
535
536   // O(h)
537   Node *inorder_predecessor(Node *node) const {
538     if (node == nullptr)
539       throw std::invalid_argument("Node is nullptr");
540     if (node->left != nullptr)
541       return rightmost_child(node->left);
542     if (node == root)
543       return nullptr;
544     Node *p = node->parent;
545     while (p != nullptr && node == p->left) {
546       if (p == root)
547         return nullptr;
548       node = p;
549       p = p->parent;
550     }
551     return p;
552   }
553
554   // O(h)
```

```
555   virtual Node *ordered_after(const T &x) const {
556     Node *result = nullptr;
557     Node *node = root;
558     while (node != nullptr) {
559       if (comp(x, node->value)) {
560         result = node;
561         node = node->left;
562       } else
563         node = node->right;
564     }
565     return result;
566   }
567
568   // O(h)
569   virtual Node *ordered_not_before(const T &x) const {
570     Node *result = nullptr;
571     Node *node = root;
572     while (node != nullptr) {
573       if (!comp(node->value, x)) {
574         result = node;
575         node = node->left;
576       } else
577         node = node->right;
578     }
579     return result;
580   }
581
582   // O(h)
583   virtual Node *ordered_before(const T &x) const {
584     Node *result = nullptr;
585     Node *node = root;
586     while (node != nullptr) {
587       if (comp(node->value, x)) {
588         result = node;
589         node = node->right;
590       } else
591         node = node->left;
592     }
593     return result;
594   }
595
596   // O(h)
597   virtual Node *ordered_not_after(const T &x) const {
598     Node *result = nullptr;
599     Node *node = root;
600     while (node != nullptr) {
601       if (!comp(x, node->value)) {
602         result = node;
603         node = node->right;
604       } else
605         node = node->left;
606     }
607     return result;
608   }
609
610   // Return node, inserted
611   // O(h)
612   virtual std::pair<Node *, bool> insert(const T &x) {
613     if (root == nullptr) {
614       root = new Node(x);
615       log("Insert:\n");
616       log_forest();
```

```
617       return {root, true};
618     }
619     Node *node = root;
620     while (true) {
621       if (comp(x, node->value)) {
622         if (node->left == nullptr) {
623           node->left = new Node(x, node);
624           log("Insert:\n");
625           log_forest();
626           return {node->left, true};
627         }
628         node = node->left;
629       } else if (comp(node->value, x)) {
630         if (node->right == nullptr) {
631           node->right = new Node(x, node);
632           log("Insert:\n");
633           log_forest();
634           return {node->right, true};
635         }
636         node = node->right;
637       } else
638         return {node, false};
639     }
640   }
641
642   // Return rebalance_start
643   // O(h)
644   virtual Node *remove_node(Node *node) {
645     if (node == nullptr)
646       throw std::invalid_argument("Node is nullptr");
647     if (node->left == nullptr) {
648       return replace(node, node->right, true);
649     }
650     if (node->right == nullptr) {
651       return replace(node, node->left, true);
652     }
653     Node *succ = inorder_successor(node);
654     Node *rebalance_start;
655     if (succ->parent != node) {
656       rebalance_start = succ->parent;
657       replace(succ, succ->right);
658       succ->right = node->right;
659       node->right->parent = succ;
660     } else
661       rebalance_start = succ;
662     replace(node, succ);
663     succ->left = node->left;
664     succ->left->parent = succ;
665     delete node;
666     log("Remove:\n");
667     log_forest();
668     return rebalance_start;
669   }
670
671   // Return rebalance_start, removed
672   // O(h)
673   virtual std::pair<Node *, bool> remove(const T &x) {
674     auto [node, found] = search(x);
675     if (!found)
676       return {node, false};
677     return {remove_node(node), true};
678   }
```

```
};

template <class T>
class AVLTreeNode : public BaseBinaryTreeNode<T, AVLTreeNode<T>> {
public:
  using Base = BaseBinaryTreeNode<T, AVLTreeNode<T>>;
  using Base::Base;
  using Base::left;
  using Base::right;
  using value_type = T;
  using node_type = Base::node_type;

  mutable int height = 1; // leaf 1

  static int get_height(AVLTreeNode *node) noexcept {
    return node == nullptr ? 0 : node->height;
  }

  static int bf(AVLTreeNode *node) {
    return get_height(node->left) - get_height(node->right);
  }

  void update_height() const {
    height = 1 + std::max(get_height(left), get_height(right));
  }
};

template <class T, class Node = AVLTreeNode<T>, class Compare = std::less<T>>
class AVLTree : public BST<T, Node, Compare> {
public:
  using node_type = Node;
  using Base = BST<T, Node, Compare>;
  using Base::Base;
  using Base::log;
  using Base::log_forest;
  using Base::root;

  AVLTree() : Base() {};
  AVLTree(const AVLTree &) = delete;
  AVLTree &operator=(const AVLTree &) = delete;

  AVLTree(AVLTree &&) = default;
  AVLTree &operator=(AVLTree &&) = default;

  Node *rotate_left(Node *node) const override {
    Node *child = Base::rotate_left(node);
    node->update_height();
    child->update_height();
    return child;
  }

  Node *rotate_right(Node *node) const override {
    Node *child = Base::rotate_right(node);
    node->update_height();
    child->update_height();
    return child;
  }

  // Rebalance until rebalanced once or parent == nullptr
  // up to outer tree if root->parent != nullptr
  virtual Node *rebalance(Node *node) const {
    node->update_height();
```

```
741     int bf = Node::bf(node);
742     if (bf > 1) {
743       Node *child = node->left;
744       if (Node::bf(child) < 0) { // LR
745         log("LR case left-then-right double rotation:\n\n");
746         rotate_left(child);
747       } else // LL
748         log("LL case right single rotation:\n\n");
749       return rotate_right(node); // LL, LR
750     }
751     if (bf < -1) {
752       Node *child = node->right;
753       if (Node::bf(child) > 0) { // RL
754         log("RL case right-then-left double rotation:\n\n");
755         rotate_right(child);
756       } else // RR
757         log("RR case left single rotation:\n\n");
758       return rotate_left(node); // RL, RR
759     }
760     if (node->parent == nullptr)
761       return node;
762     return rebalance(node->parent);
763   }
764
765   std::pair<Node *, bool> insert(const T &x) override {
766     auto [node, inserted] = Base::insert(x);
767     if (inserted && node->parent != nullptr)
768       rebalance(node->parent);
769     return {node, inserted};
770   }
771
772   Node *remove_node(Node *node) override {
773     Node *rebalance_start = Base::remove_node(node);
774     Node *cur = rebalance_start;
775     while (cur != nullptr) {
776       cur = rebalance(cur)->parent;
777     }
778     return rebalance_start;
779   }
780 };
781
782 template <class T, class Node = BinaryTreeNode<T>, class Compare = std::less<T>>
783 class SplayTree : public BST<T, Node, Compare> {
784 public:
785   using node_type = Node;
786   using Base = BST<T, Node, Compare>;
787   using Base::Base;
788   using Base::comp;
789   using Base::is_left_child;
790   using Base::log;
791   using Base::log_forest;
792   using Base::oss;
793   using Base::root;
794   using Base::rotate_parent;
795
796 protected:
797   using Base::replace;
798
799 public:
800   SplayTree() : Base() {};
801   SplayTree(const SplayTree &) = delete;
802   SplayTree &operator=(const SplayTree &) = delete;
```

```
803
804   SplayTree(SplayTree &&) = default;
805   SplayTree &operator=(SplayTree &&) = default;
806
807   // O(h) worst case, O(log n) amortized
808   virtual Node *splay(Node *node) const {
809     if (node == nullptr)
810       return nullptr;
811     while (node != root) {
812       Node *parent = node->parent;
813       Node *grandparent = parent->parent;
814       // Zig
815       if (parent == root) {
816         log("Zig step:\n\n");
817         rotate_parent(node);
818         // Zig-zig
819       } else if ((parent->left == node && grandparent->left == parent) ||
820                  (parent->right == node && grandparent->right == parent)) {
821         log("Zig-zig step:\n\n");
822         rotate_parent(parent);
823         rotate_parent(node);
824       }
825       // Zig-zag
826       else {
827         log("Zig-zag step:\n\n");
828         rotate_parent(node);
829         rotate_parent(node);
830       }
831     }
832     return node;
833   }
834
835   Node *find_min(Node *node) const override {
836     return splay(Base::find_min(node));
837   }
838
839   Node *find_max(Node *node) const override {
840     return splay(Base::find_max(node));
841   }
842
843   Node *find_min() const override { return find_min(root); }
844   Node *find_max() const override { return find_max(root); }
845
846   Node *ordered_after(const T &x) const override {
847     return splay(Base::ordered_after(x));
848   }
849
850   Node *ordered_not_before(const T &x) const override {
851     return splay(Base::ordered_not_before(x));
852   }
853
854   Node *ordered_not_after(const T &x) const override {
855     return splay(Base::ordered_not_after(x));
856   }
857
858   Node *ordered_before(const T &x) const override {
859     return splay(Base::ordered_before(x));
860   }
861
862   std::pair<Node *, bool> search(const T &x) const override {
863     auto [result, second] = Base::search(x);
864     return {splay(result), second};
```

```
865   }
866
867   std::pair<Node *, bool> insert(const T &x) override {
868     auto [result, second] = Base::insert(x);
869     return {splay(result), second};
870   }
871
872   virtual void join_right(Node *other) {
873     if (other == nullptr)
874       return;
875     if (root == nullptr) {
876       root = other;
877       other->parent = nullptr;
878     } else {
879       find_max();
880       root->right = other;
881       other->parent = root;
882     }
883     log("Join right:\n");
884     log_forest();
885   }
886
887   virtual void join_left(Node *other) {
888     if (other == nullptr)
889       return;
890     if (root == nullptr) {
891       root = other;
892       other->parent = nullptr;
893     } else {
894       find_min();
895       root->left = other;
896       other->parent = root;
897     }
898     log("Join left:\n");
899     log_forest();
900   }
901
902   // Not return rebalance_start but always return nullptr
903   Node *remove_node(Node *node) override {
904     if (node == nullptr)
905       throw std::invalid_argument("Node is nullptr");
906     Node *victim = splay(node);
907     Node *right = victim->right;
908     if (victim->left != nullptr) {
909       replace(victim, victim->left, true);
910       if (right != nullptr)
911         right->parent = nullptr;
912       join_right(right);
913     } else
914       replace(victim, right, true);
915     return nullptr;
916   }
917
918 protected:
919   virtual SplayTree split_right(Node *other_root) {
920     if (other_root == nullptr)
921       return SplayTree(nullptr, comp, oss);
922     replace(other_root, other_root->left);
923     other_root->parent = nullptr;
924     other_root->left = nullptr;
925     SplayTree result = SplayTree(other_root, comp, oss);
926     log_forest();
```

```
927     result.log_forest();
928     return result;
929   }
930
931   virtual SplayTree split_left(Node *other_root) {
932     if (other_root == nullptr)
933       return SplayTree(nullptr, comp, oss);
934     replace(other_root, other_root->right);
935     other_root->parent = nullptr;
936     other_root->right = nullptr;
937     SplayTree result = SplayTree(other_root, comp, oss);
938     log_forest();
939     result.log_forest();
940     return result;
941   }
942
943 public:
944   virtual SplayTree split_after(const T &x) {
945     log("Split after:\n\n");
946     return split_right(ordered_after(x));
947   }
948
949   virtual SplayTree split_not_before(const T &x) {
950     log("Split not before:\n\n");
951     return split_right(ordered_not_before(x));
952   }
953
954   virtual SplayTree split_before(const T &x) {
955     log("Split before:\n\n");
956     return split_left(ordered_before(x));
957   }
958
959   virtual SplayTree split_not_after(const T &x) {
960     log("Split not after:\n\n");
961     return split_left(ordered_not_after(x));
962   }
963
964   // O(h) worst case, O(log n) amortized
965   virtual std::pair<Node *, bool> top_down_splay_search(const T &x) const {
966     if (root == nullptr)
967       return {nullptr, false};
968     bool found = false;
969     Node *old_parent = root->parent;
970     signed char old_side = is_left_child(root);
971     Node *L = nullptr;
972     Node *l = nullptr;
973     Node *R = nullptr;
974     Node *r = nullptr;
975     Node *m = root;
976     while (true) {
977       if (comp(x, m->value)) {
978         if (m->left == nullptr)
979           break;
980         if (comp(m->left->value, x)) {
981           if (R == nullptr) {
982             R = m;
983             r = m;
984             m = m->left;
985             R->left = nullptr;
986             m->parent = nullptr;
987           } else {
988             r->left = m;
```

```
            m->parent = r;
            r = m;
            m = m->left;
            m->parent = nullptr;
            log("Right tree:\n");
            log_forest(R);
          }
        } else
          m = rotate_parent(m->left);
      } else if (comp(m->value, x)) {
        if (m->right == nullptr)
          break;
        if (comp(x, m->right->value)) {
          if (L == nullptr) {
            L = m;
            l = m;
            m = m->right;
            L->right = nullptr;
            m->parent = nullptr;
          } else {
            l->right = m;
            m->parent = l;
            l = m;
            m = m->right;
            m->parent = nullptr;
            log("left tree:\n");
            log_forest(L);
          }
        } else
          m = rotate_parent(m->right);
      } else {
        found = true;
        break;
      }
    }
    root = m;
    if (L != nullptr) {
      if (m->left != nullptr) {
        l->right = m->left;
        m->left->parent = l;
        log("left tree:\n");
        log_forest(L);
      }
      m->left = L;
      L->parent = m;
    }
    if (R != nullptr) {
      if (m->right != nullptr) {
        r->left = m->right;
        m->right->parent = r;
        log("Right tree:\n");
        log_forest(R);
      }
      m->right = R;
      R->parent = m;
    }
    root->parent = old_parent;
    switch (old_side) {
    case 1:
      old_parent->left = root;
      break;
    case 0:
```

```
      old_parent->right = root;
      break;
    }
    return {root, found};
  }
};
```